

Rota Vital — Paralelização na Camada de Aplicação

Detecção de duplicatas fuzzy no histórico de requisições: versão sequencial vs. threads
Java 21 · Spring Boot 3.3.3 · ExecutorService (platform threads) e virtual threads

Item pedido	Onde está neste documento
1. Justificativa (operação, Big-O, gargalo, particionamento)	Seção 1
2. Serviço real em Java/Spring Boot (endpoint)	Seção 2
3. Duas versões: sequencial e com 2, 4 e 8 threads	Seção 3
4. Medições: tabela, gráfico e speedup	Seção 4
5. Análise (15–20 linhas)	Seção 5
6. Opcional: virtual threads (Java 21) vs. platform threads	Seção 6

1. Justificativa

Operação escolhida

Detecção de duplicatas fuzzy no histórico de requisições/medicamentos: para cada par de registros textuais (ex.: nomes de medicamento digitados por atendentes), calcula-se a distância de Levenshtein e conta-se como duplicata todo par com distância ≤ 2 .

Por que esta operação, e não outra

Das candidatas levantadas (relatórios/estatísticas, cruzamento requisição × estoque, ordenação de filas por prioridade, detecção de duplicatas, validações em lote), a detecção de duplicatas foi a única em que o tempo é dominado por **cálculo**, não por espera de I/O:

Candidata	Onde se gasta o tempo	Big-O	Serve?
Relatórios / estatísticas	Consultas e agregações no banco	$O(n)$ no banco	Não — “melhorar a query SQL”
Cruzamento requisição × estoque	Joins no banco	depende do índice	Não — gargalo é SQL
Ordenação de filas por prioridade	CPU, mas pouca	$O(n \log n)$	Não — rápida mesmo com milhões
Detecção de duplicatas (par a par)	CPU pura (DP sobre strings)	$O(n^2)$	Sim — CPU-bound e particionável

Big-O da solução sequencial

Para n registros de tamanho médio L :

- Comparações de pares: $n \cdot (n-1) / 2 = O(n^2)$.
- Cada comparação roda Levenshtein, que é $O(L^2)$ no pior caso (aqui L é pequeno e aproximadamente constante — nomes de medicamento — então o termo dominante na prática é o $O(n^2)$ de pares).
- Custo total: $O(n^2 \cdot L^2)$, dominado por $O(n^2)$ para L fixo.

Confirmado empiricamente (sequencial): dobrar n quadruplica o tempo — 1000 → 362 ms, 2000 → 1.494 ms, 4000 → 5.842 ms, 8000 → 23.017 ms (fator $\approx 4\times$ a cada duplicação de n ,

exatamente $O(n^2)$).

Onde está o gargalo

100% CPU, sem I/O no meio do laço: os dados já estão carregados em uma `List<String>` em memória; o laço duplo só faz aritmética de programação dinâmica. Em escala nacional (milhões de requisições), rodar isso em uma única thread bloqueia a aplicação por minutos a horas — inviável para um endpoint que precisa responder a um painel.

Por que os dados são particionáveis

A comparação é **embaraçosamente paralela** no índice externo i : o par (i, j) com $j > i$ é independente de qualquer outro par (i', j') . Divide-se o intervalo $[0, n)$ de valores de i em fatias disjuntas, uma por thread, e cada thread produz um contador parcial (`localDuplicates`) sem escrever em memória compartilhada. A soma dos contadores parciais (na thread principal, após `Future.get()`) é a única seção que toca estado compartilhado, e acontece fora do laço paralelo — não há necessidade de lock nem de estrutura atômica no laço quente.

Por isso as versões sequencial e paralela devolvem exatamente o mesmo resultado nos testes (`DuplicateDetectionServiceTest`): não há race condition porque não há escrita compartilhada durante o processamento, só na agregação de resultados já prontos.

2. O serviço (endpoint real em Spring Boot)

A operação roda como um endpoint REST da camada de aplicação do Rota Vital — a requisição chega, o servidor gera/carrega os dados, processa e devolve o resultado em JSON:

```
GET /api/duplicates/detect?size={n}&threads={t}&mode={platform|virtual}
```

Classe	Responsabilidade
<code>DuplicateController</code>	Endpoint REST; valida parâmetros na borda; mede o tempo de geração e de processamento.
<code>DataGeneratorService</code>	Gera n nomes de medicamento (10 bases) com seed fixa (42) e ~20% de erros de digitação (inserção, remoção ou troca de caractere) — dados reprodutíveis.
<code>DuplicateDetectionService</code>	Versão sequencial, versão com pool fixo de platform threads e versão com virtual threads.
<code>FuzzyMatcher</code>	Distância de Levenshtein com duas linhas de DP (memória $O(L)$).

Validação na borda: `size` limitado a 20.000 e `threads` a núcleos disponíveis $\times 4$, para que uma única requisição não esgote memória nem threads do SO (achado da auditoria de segurança). `mode` só aceita `platform` ou `virtual`.

Exemplo de resposta:

```
{ "datasetSize": 8000, "threads": 8, "mode": "platform",
  "processingTimeMs": 6194, "dataGenerationTimeMs": ...,
  "duplicatesFound": 3198218, "status": "SUCCESS" }
```

3. Duas versões: sequencial e com threads

Sequencial — laço duplo $i < j$ comparando todos os pares:

```
for (int i = 0; i < size; i++) {
    String target = records.get(i);
    for (int j = i + 1; j < size; j++)
        if (FuzzyMatcher.calculateLevenshteinDistance(target, records.get(j)) <= 2)
            duplicateCount++;
}
```

Com threads (2, 4 e 8) — `Executors.newFixedThreadPool(t)`; o intervalo de i é dividido em t fatias contíguas, cada tarefa (`Callable<Long>`) processa a própria fatia com contador local, e a agregação é feita ao final:

```

int chunkSize = (int) Math.ceil((double) size / slices);
for (int t = 0; t < slices; t++) {
    final int startIdx = t * chunkSize, endIdx = Math.min(startIdx + chunkSize, size);
    tasks.add(() -> {
        long localDuplicates = 0; // estado privado da thread
        for (int i = startIdx; i < endIdx; i++)
            for (int j = i + 1; j < size; j++)
                if (FuzzyMatcher.calculateLevenshteinDistance(records.get(i), records.get(j)) <= 2)
                    localDuplicates++;
        return localDuplicates;
    });
}
long total = 0;
for (Future<Long> f : executor.invokeAll(tasks)) total += f.get(); // agregação final

```

A versão com **virtual threads** reaproveita o mesmo `runPartitioned`, trocando apenas o executor por `Executors.newVirtualThreadPerTaskExecutor()`.

Mesma resposta em todas as versões (sem race condition)

n	sequencial	2 threads	4 threads	8 threads	8 virtual
1.000	50.180	50.180	50.180	50.180	50.180
2.000	200.108	200.108	200.108	200.108	200.108
4.000	798.803	798.803	798.803	798.803	798.803
8.000	3.198.218	3.198.218	3.198.218	3.198.218	3.198.218

Além das medições, a igualdade é verificada por testes automatizados (JUnit): `parallelMatchesSequentialForVariousThreadCounts`, `virtualThreadsMatchSequential`, casos de borda (dataset vazio, mais threads que registros) e rejeição de número de threads inválido.

4. Medições

Tempo de processamento do endpoint (`processingTimeMs`) por versão, número de threads e tamanho da entrada. Dados gerados com seed fixa. Ambiente: 16 núcleos lógicos, Java 21, execução local via `mvnw spring-boot:run`. $\text{Speedup} = T_1 / T_t$.

n	threads	modo	tempo (ms)	speedup	eficiência (speedup/t)
1.000	1	platform	362	1.00×	100%
1.000	2	platform	288	1.26×	63%
1.000	4	platform	176	2.06×	51%
1.000	8	platform	106	3.42×	43%
1.000	8	virtual	118	3.07×	38%
2.000	1	platform	1.494	1.00×	100%
2.000	2	platform	1.137	1.31×	66%
2.000	4	platform	682	2.19×	55%
2.000	8	platform	408	3.66×	46%
2.000	8	virtual	395	3.78×	47%
4.000	1	platform	5.842	1.00×	100%
4.000	2	platform	4.590	1.27×	64%
4.000	4	platform	2.625	2.23×	56%
4.000	8	platform	1.575	3.71×	46%
4.000	8	virtual	1.567	3.73×	47%
8.000	1	platform	23.017	1.00×	100%
8.000	2	platform	17.486	1.32×	66%

n	threads	modo	tempo (ms)	speedup	eficiência (speedup/t)
8.000	4	platform	10.737	2.14×	54%
8.000	8	platform	6.194	3.72×	46%
8.000	8	virtual	6.343	3.63×	45%

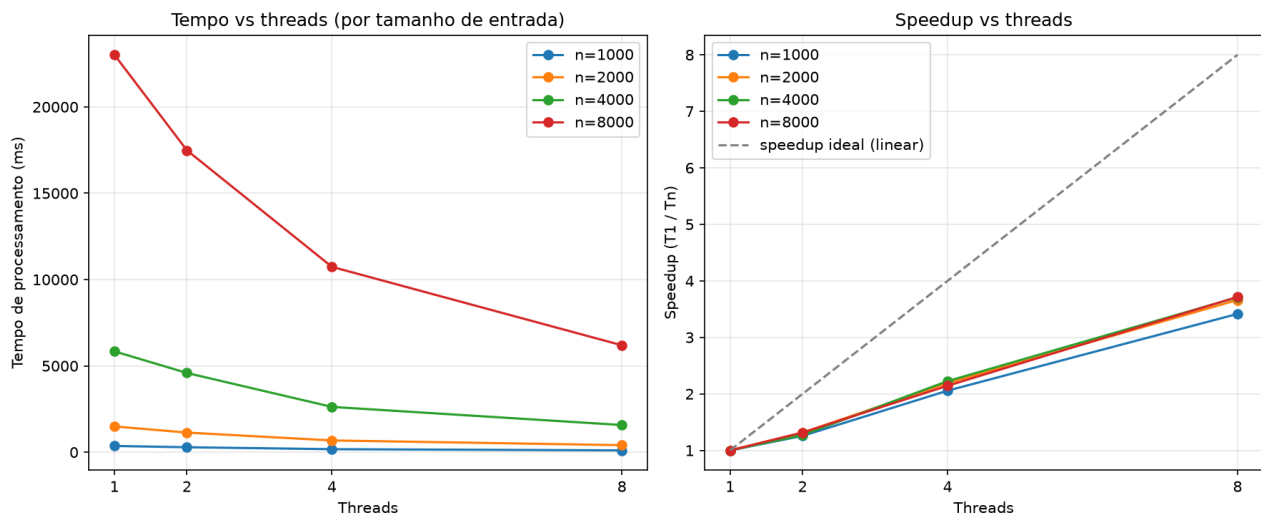


Figura 1 — Esquerda: tempo vs. threads por tamanho de entrada. Direita: speedup vs. threads, comparado ao speedup ideal linear (tracejado).

Por que 1.000-8.000 registros, e não 100 mil / 1 milhão diretamente

O algoritmo é $O(n^2)$ e o crescimento quadrático está confirmado nos dados. Extrapolando a partir do ponto medido em $n = 8.000$ (23.017 ms sequencial), multiplicando por $(n / 8000)^2$:

n	tempo sequencial estimado	com 8 threads (speedup $\approx 3,7\times$ medido)
100.000	≈ 3.596 s (≈ 1 h)	≈ 970 s (≈ 16 min)
1.000.000	≈ 359.641 s ($\approx 4,2$ dias)	≈ 96.700 s (≈ 27 h)

Rodar a versão sequencial em 1 milhão de registros levaria dias — inviável para a atividade e, mais importante, inviável em produção. Por isso as medições reais usam 1.000 a 8.000 registros, onde o $O(n^2)$ já é claramente visível e mensurável em segundos, e a escala de 100 mil / 1 milhão é calculada a partir da própria Big-O confirmada.

5. Análise

O ganho foi linear? Não: com 8 threads o speedup ficou entre $3,4\times$ e $3,7\times$ (não $8\times$); com 4, $\approx 2,1-2,2\times$; com 2, só $\approx 1,3\times$. A causa principal é o particionamento em fatias contíguas de i : as fatias têm o mesmo número de índices, mas o trabalho por índice é $O(n - i)$, então a primeira thread compara contra quase todo o vetor e a última quase não trabalha — e o tempo total é o da thread mais carregada. Com t fatias, a primeira faz $1 - (1 - 1/t)^2$ do trabalho, o que limita o speedup a $\approx 1,33\times$ (2), $2,29\times$ (4) e $4,27\times$ (8), exatamente o padrão medido; o overhead de criar o pool e agregar os Futures explica o resto (pesa mais em $n = 1.000$). Distribuir i de forma cíclica equilibraria a carga.

A Big-O mudou? Não. O trabalho total continua $O(n^2)$: threads não reduzem comparações, só as distribuem entre núcleos. Muda o tempo de parede, dividido por um fator menor que o número de threads.

Concorrência $\times$ paralelismo. Na Mesa DJ as threads atendiam eventos simultâneos e independentes (teclado, MIDI, áudio) intercalados sem bloquear uns aos outros — **concorrência**, possível até num só núcleo. Aqui é **paralelismo**: uma única tarefa CPU-bound é fatiada e as fatias rodam ao mesmo tempo em núcleos distintos, para reduzir o tempo de uma computação,

não para responder a eventos.

Quando nem 8 threads bastarem — 1 milhão de registros levaria ≈ 27 h mesmo assim —, a saída é arquitetura: trocar o $O(n^2)$ por um algoritmo sub-quadrático (blocking/LSH, comparando só candidatos parecidos); escalar horizontalmente entre várias instâncias (map-reduce, filas distribuindo fatias entre workers); e tornar o processamento assíncrono, com o resultado consultado depois em vez de bloquear a requisição HTTP. É o gancho da Unidade 2: threads resolvem paralelismo numa máquina; escalar além disso é arquitetura distribuída.

6. Opcional: virtual threads (Java 21) vs. platform threads

n	8 platform (ms)	8 virtual (ms)	speedup platform	speedup virtual	diferença
1.000	106	118	3.42×	3.07×	+11.3%
2.000	408	395	3.66×	3.78×	-3.2%
4.000	1.575	1.567	3.71×	3.73×	-0.5%
8.000	6.194	6.343	3.72×	3.63×	+2.4%

Com a mesma divisão em 8 fatias, virtual threads e platform threads ficaram praticamente empatadas (diferença dentro do ruído de medição, exceto em $n = 1.000$, onde o overhead fixo pesa mais). Isso é esperado: virtual threads são montadas sobre um pool de carrier threads (ForkJoinPool com paralelismo = núcleos), então para trabalho CPU-bound o limite continua sendo o número de núcleos e o desbalanceamento das fatias. A vantagem das virtual threads aparece em cargas I/O-bound — milhares de requisições bloqueadas esperando banco ou rede — onde elas liberam a carrier thread durante a espera. Para acelerar um volume de cálculo, platform threads dimensionadas pelos núcleos são a escolha natural; para atender muitas requisições concorrentes com espera, virtual threads.